

A TECHNICAL REPORT OF THE ANTIGRAVITY RESEARCH SERIES

DJIA ALGORITHMIC BACKTESTER

Development, Multi-Agent Architecture,
and Mathematical Blueprint

STRATEGY DESIGN · AGENTIC ENGINEERING · BACKTESTING

“From a natural-language investment idea to an interactive simulation engine: a technical account of how an algorithmic strategy is designed, built, tested, and explained.”

ALEJANDRO REYNOSO

JUDGE BUSINESS SCHOOL · UNIVERSITY OF CAMBRIDGE · JULY 13, 2026

Executive abstract

This report provides a detailed overview of the design, engineering history, and architectural implementation of the Dow Jones Industrial Average (DJIA) Algorithmic Strategy Backtester. Starting with a brief natural language prompt, this system was developed through an iterative process involving a human supervisor and autonomous AI agents. The document describes the multi-agent collaboration between the core agent, background task threads, and the browser subagent. It traces the continuous improvement loops that led to a dynamic rebalancing feed and timeline playback. Additionally, the appendix serves as a detailed guide for developers to implement the Antigravity algorithmic trading models, complete with mathematical formulations and JavaScript code implementations.

Contents

1	Introduction & Human Prompting Narrative	3
1.1	Phase 1: Initial Core Request	3
1.2	Phase 2: Transition to Dynamic Playback	3
1.3	Phase 3: Chronological Timeline Feed	3
2	Google Antigravity: Execution Loop and Platform Mechanics	5
2.1	Unified Surfaces and TUI/GUI Channels	5
2.2	The Reactive Agentic Execution Loop	5
2.3	Isolation, Permissions, and Sandboxing	5
2.4	Skills & Customizations Architecture	6
3	Agents Involved & Multi-Agent Architecture	7
4	Actions Taken by the Agents	8
5	Iterative Software Development & Refactoring	9
5.1	Version 1.0: Static Batch Engine	9
5.2	Version 2.0: Caching Playback Engine	9
5.3	Version 3.0: Chronological Timeline Feed	9
6	Final Result & System Execution Model	10
A	Algorithmic Trading Model Implementation Guide	11
A.1	Mathematical Formulations	11
A.2	JavaScript Code Implementations	14

REPORT SECTION 1

Introduction & Human Prompting Narrative

The inception, development, and refinement of the DJIA Algorithmic Backtester were driven by natural language prompts from the human user. The project progressed through three distinct phases:

1.1 Phase 1: Initial Core Request

The human operator initiated the project with the prompt:

Human prompt

"i want to have an algorithmic strategy to invest in the components of the dow jones"

This prompt was analyzed by the Antigravity core agent, identifying three key components:

- ▶ **Data Universe:** The 30 current stocks comprising the Dow Jones Industrial Average.
- ▶ **Strategy Space:** Algorithmic selection and rebalancing methods (e.g., Dogs of the Dow, Momentum, Mean Reversion).
- ▶ **Evaluation Engine:** A backtesting simulator to calculate historical returns, drawdowns, and risks.

The core agent created an implementation plan (`implementation_plan.md`) and a task list (`task.md`), which the user reviewed and approved.

1.2 Phase 2: Transition to Dynamic Playback

After the static codebase was built, the user requested an interactive animation of the backtest:

Human prompt

"now, i want a more interactive visualization showing the dynamic change in composition of the portfolio...the entrants and the exiting stocks ... show a dynamic building of the backtesting curve"

This prompt required a shift from static batch processing to stateful daily simulation. The core agent modified the data structures to cache daily snapshots, added playback controls (play, pause, reset, speed), and built an entrant/exit card system to show which stocks entered and left the portfolio.

1.3 Phase 3: Chronological Timeline Feed

The final request aimed to capture the entire rebalancing history:

Human prompt

"i want to see the changes in the porfolio composition at every rbalancing, in a dynamic display"

The single-event panel was replaced with a scrollable timeline feed. As playback runs, new cards are appended to the feed, building a scrollable record of entrants and exits for all rebalance events since 2015.

REPORT SECTION 2

Google Antigravity: Execution Loop and Platform Mechanics

To understand the engineering behind this backtester, it is necessary to examine how the Google Antigravity platform operates. Antigravity is an AI-first software engineering ecosystem designed by Google DeepMind, structured to orchestrate complex development cycles autonomously.

2.1 Unified Surfaces and TUI/GUI Channels

The platform exposes three interaction surfaces:

- ▶ **Antigravity 2.0 Desktop Application:** An Electron-based desktop surface featuring a side-by-side split chat canvas, project selector, and an Auxiliary Pane to monitor subagent threads, background tasks, files changed, and active terminals.
- ▶ **Antigravity TUI CLI (agy):** A terminal-based Text User Interface. It operates via JSON configurations stored in `~/.gemini/antigravity-cli/settings.json` and uses keyboard shortcuts (like ESC to toggle menus or arrow keys for command logs).
- ▶ **Antigravity SDK:** A Python library for leasing virtual agent instances programmatically, invoking remote tools, and orchestrating multi-agent teamwork pipelines.

2.2 The Reactive Agentic Execution Loop

Antigravity operates on a reactive event-driven cycle rather than a polling loop. The execution flow is modeled as follows:

1. **Ingestion & Planning:** The agent receives user input, queries context pools (@ mentions, rule structures), and plans its changes.
2. **Tool Call Proposal:** The agent issues a structured JSON tool call (e.g., `run_command`, `write_to_file`).
3. **Halt and Yield (Idle State):** The execution of the agent freezes. No CPU cycles are consumed while the system awaits human approval or background command completions.
4. **Reactive Event Wakeup:** When a background command finishes or the user clicks “Proceed”, the system fires a JSON-RPC notification. The dispatcher wakes the agent, injecting the outputs directly back into its context window.

2.3 Isolation, Permissions, and Sandboxing

Security policies protect the host system during agent runs:

- ▶ **Workspace Restrictions:** By default, file actions are restricted to the active workspace directory. Accessing external directories requires a permission request.
- ▶ **Command Policy Review:** Terminal commands are executed inside a sandboxed environment. Execution modes can be set to `always-proceed`, `request-review`, or `strict` (relying

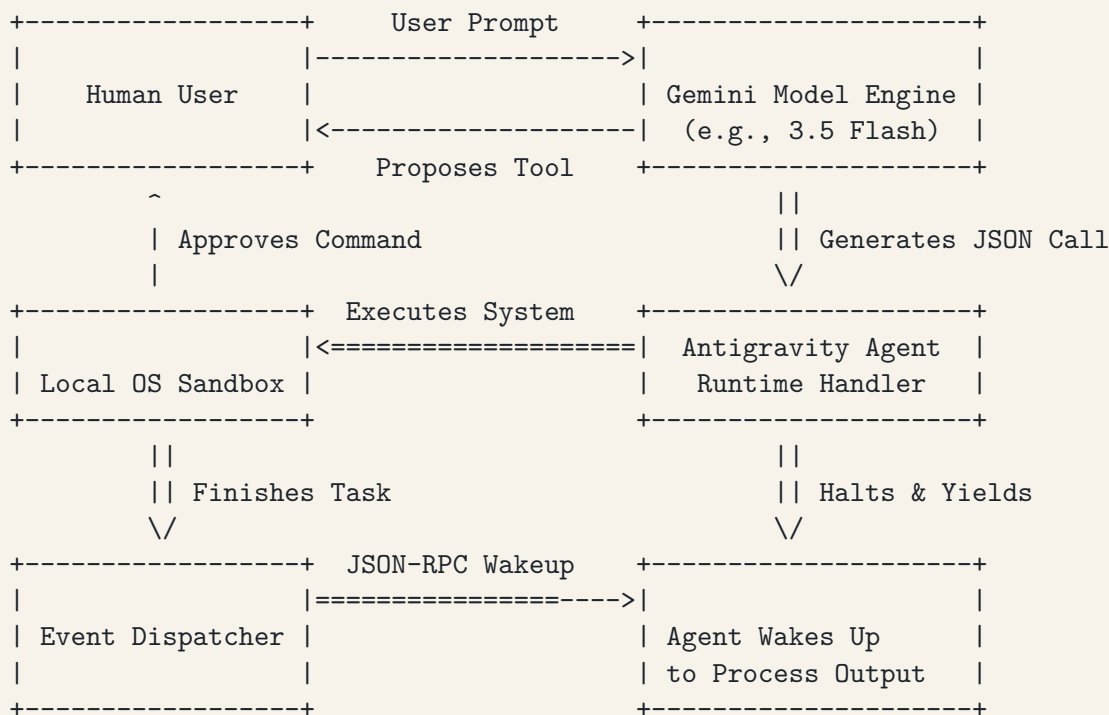


Figure 1. Reactive Agentic Loop Cycle

on a trusted command whitelist).

- **Network Domain Whitelist:** Prevents unauthorized data egress. The agent's web browsing tools can only navigate to domains explicitly permitted by the project settings.

2.4 Skills & Customizations Architecture

Antigravity discovers customizations dynamically from two roots: the Global Customizations Root (~/.gemini/config) and the Workspace Customizations Root (.agents/).

- **Skills (Directory Packages):** Self-contained folders containing a SKILL.md instruction file with YAML frontmatter, along with optional helper scripts, templates, and reference materials. These are matched dynamically based on the name and description keys.
- **Rules (Behavioral Modifiers):** Appended to the AGENTS.md markdown file. They define project-specific coding guidelines, constraints, and instructions that are automatically appended to the agent's system instructions.
- **Model Context Protocol (MCP):** Custom services exposed via local JSON-RPC pipes. They allow developers to expose custom compilers, debuggers, or specialized APIs to the agent's toolbelt.

REPORT SECTION 3

Agents Involved & Multi-Agent Architecture

The development and verification of the backtester relied on a coordinated multi-agent structure:

- ▶ **Antigravity Core Agent:** The primary coordinator. It processes user prompts, makes architectural design choices, plans execution paths, manages the filesystem, writes source code, and coordinates subagents.
- ▶ **Browser Subagent:** A specialized agent designed to verify web layouts. It is equipped with tools to launch a Chromium instance, navigate pages, read the DOM tree, type input, click elements, and capture screenshots. During Phase 1 verification, this subagent reported a local Chrome execution failure because the Windows environment restricts the sandbox mode. Antigravity core handled this failure by pivoting to automated unit testing and manual user inspection.
- ▶ **Task Executors (OS/Process Layer):** Background execution nodes managing parallel execution commands, such as installing packages via pip or serving local web assets via Python's HTTP server.

REPORT SECTION 4

Actions Taken by the Agents

The agents took the following steps to build and verify the system:

1. **Path & Runtime Analysis:** Proposed a environment check to locate Python. Discovered Python 3.14.3 installed in the local user path: `C:/Users/areyn/AppData/Local/Python/pythoncore-3.14-64/python.exe`.
2. **Asynchronous Dependency Ingestion:** Proposed a background command to install dependencies:

```
python -m pip install pandas yfinance
```

The task system executed this, installing pandas 3.0.3, yfinance 1.5.1, and numpy 2.5.1.

3. **Data Universe Harvesting:** Wrote and executed `download_data.py`. The task downloaded daily prices and dividend events for all 30 DJIA components (from 2015-01-01 to 2026-07-13) and saved them to `djia_history.json`.
4. **Dashboard Implementation:** Coded the front-end layout (`index.html`), design system (`style.css`), and backtesting logic (`app.js`).
5. **Validation:** Created `verify_backtest.py` to check data integrity, confirming 2,897 days of stock history with no NaNs or negative prices.
6. **Spawning Local Delivery:** Started a local web server:

```
python -m http.server 8000
```

which serves the backtester on port 8000.

REPORT SECTION 5

Iterative Software Development & Refactoring

The system went through three major software versions:

5.1 Version 1.0: Static Batch Engine

The initial implementation ran backtests on page load. It calculated cumulative statistics and rendered final static plots. It did not track daily asset allocation changes or individual trade histories.

5.2 Version 2.0: Caching Playback Engine

Refactored the simulator to record daily states. This allowed the system to:

- ▶ Track a daily `dailySnapshots` array with asset shares, cash, and total valuation.
- ▶ Update the `Chart.js` dataset slices on every tick of a timer loop.
- ▶ Detect when the playback index crossed a rebalancing index, calculating entrants and exits relative to the previous rebalance date.

5.3 Version 3.0: Chronological Timeline Feed

Replaced the single-event display with a scrollable feed container (`#timelineFeed`).

- ▶ **Dynamic Appending:** During playback, when a rebalance date is crossed, a new event card is created and prepended to the top of the feed container.
- ▶ **Static Rendering:** When running a new backtest, the full timeline is rendered immediately. Prepending chronological cards ensures the latest rebalance event is always visible at the top.
- ▶ **Animation Optimization:** Calling `chart.update('none')` disabled default transitions during playback. This prevented `Chart.js` from lagging on daily data updates.

REPORT SECTION 6

Final Result & System Execution Model

The final application resides in a workspace directory containing five core files:

1. `download_data.py`: Downloads prices and dividends using `yfinance` and outputs `djia_history.json`.
2. `djia_history.json`: Aligned 2,897-day database of raw close prices, adjusted close prices, and dividends.
3. `index.html`: Semantic HTML5 structure with custom widgets for controls and the timeline.
4. `style.css`: Custom design system utilizing HSL variables for emerald, blue, and slate tones, glassmorphism card blur, and CSS animations.
5. `app.js`: Simulator engine, playback timeline loop, and `Chart.js` renderer.

The application runs locally using Python's built-in HTTP server:

```
python -m http.server 8000
```

Navigate to `http://localhost:8000/` to run the application.

REPORT SECTION **A**

Algorithmic Trading Model Implementation Guide

This appendix provides a step-by-step mathematical and programmatic guide to implementing the Antigravity algorithmic trading models.

A.1 Mathematical Formulations

A.1.1 Portfolio Valuation Model

Let C_t be the cash balance on trading day t . Let $S_{i,t}$ be the number of shares held of stock i on day t . Let $P_{i,t}^{adj}$ be the adjusted close price of stock i on day t (accounting for splits and dividend distributions). The total portfolio value V_t on day t is:

$$V_t = C_t + \sum_{i \in \mathcal{U}} S_{i,t} \cdot P_{i,t}^{adj} \quad (1)$$

where $\mathcal{U}$ is the universe of 30 DJIA tickers.

A.1.2 Trading Costs and Rebalancing

Rebalancing occurs at specific daily indices $t \in \mathcal{T}_{rebal}$. Let $w_{i,t}$ be the target portfolio weight of asset i on rebalance day t , where $\sum_{i \in \mathcal{U}} w_{i,t} = 1.0$. The target dollar allocation for stock i is $V_{i,t}^{target} = V_t \cdot w_{i,t}$. The current value of shares held is $V_{i,t}^{curr} = S_{i,t-1} \cdot P_{i,t}^{adj}$. The trade amount $|\Delta V_{i,t}|$ for stock i is:

$$|\Delta V_{i,t}| = |V_{i,t}^{target} - V_{i,t}^{curr}| \quad (2)$$

Given a transaction cost rate γ (e.g., 0.001 for 0.10% slippage/commission), the total transaction costs TC_t are computed as:

$$TC_t = \gamma \cdot \sum_{i \in \mathcal{U}} |\Delta V_{i,t}| \quad (3)$$

The remaining investable capital V_t^{net} is:

$$V_t^{net} = V_t - TC_t \quad (4)$$

New share quantities $S_{i,t}$ are updated as:

$$S_{i,t} = \frac{V_t^{net} \cdot w_{i,t}}{P_{i,t}^{adj}} \quad (5)$$

The cash balance becomes:

$$C_t = V_t^{net} - \sum_{i \in \mathcal{U}} S_{i,t} \cdot P_{i,t}^{adj} \quad (6)$$

A.1.3 Dogs of the Dow (Dividend Yield Selection)

For rebalancing date t , the trailing 12-month dividend yield $Y_{i,t}$ for stock i is calculated using the raw close price $P_{i,t}^{close}$ and the sum of dividends paid in the last 365 days:

$$Y_{i,t} = \frac{\sum_{\tau \in [t-365, t]} Div_{i,\tau}}{P_{i,t}^{close}} \quad (7)$$

The assets are ranked in descending order of yield:

$$Y_{(1),t} \geq Y_{(2),t} \geq \dots \geq Y_{(|\mathcal{U}|),t} \quad (8)$$

The top N stocks are selected ($\mathcal{S}_t = \{(1), (2), \dots, (N)\}$). Weights are assigned equally:

$$w_{i,t} = \begin{cases} \frac{1}{N} & \text{if } i \in \mathcal{S}_t \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

A.1.4 Momentum (Relative Strength Selection)

Let L represent the lookback period in trading days (e.g., 6 months ≈ 126 days). The momentum return $R_{i,t}^M$ is:

$$R_{i,t}^M = \frac{P_{i,t}^{adj} - P_{i,t-L}^{adj}}{P_{i,t-L}^{adj}} \quad (10)$$

An optional 200-day Simple Moving Average (SMA) filter requires:

$$P_{i,t}^{adj} > \text{SMA}_{200}(P_i^{adj})_t = \frac{1}{200} \sum_{\tau=t-199}^t P_{i,\tau}^{adj} \quad (11)$$

Stocks failing the SMA check are filtered out of the selection pool. The remaining assets are sorted in descending order of $R_{i,t}^M$, and the top N are assigned equal weights $1/N$.

A.1.5 Mean Reversion Selection

Calculated identically to Momentum, except the selection pool is sorted in ascending order of $R_{i,t}^M$ (worst performers selected).

A.1.6 CAGR (Compound Annual Growth Rate)

$$\text{CAGR} = \left(\frac{V_{end}}{V_{start}} \right)^{\frac{252}{D_{total}}} - 1 \quad (12)$$

where D_{total} is the total number of simulated trading days.

A.1.7 Annualized Volatility

Daily returns are calculated as $r_t = \frac{V_t - V_{t-1}}{V_{t-1}}$. Given the mean daily return $\bar{r}$, daily variance σ_{daily}^2 is:

$$\sigma_{daily}^2 = \frac{1}{D_{total} - 1} \sum_{t=1}^{D_{total}-1} (r_t - \bar{r})^2 \quad (13)$$

Annualized volatility σ_{annual} is:

$$\sigma_{annual} = \sqrt{\sigma_{daily}^2} \cdot \sqrt{252} \quad (14)$$

A.1.8 Sharpe Ratio

Assuming an annualized risk-free rate R_f (defaulting to 2.0%):

$$\text{Sharpe Ratio} = \frac{\text{CAGR} - R_f}{\sigma_{annual}} \quad (15)$$

A.1.9 Maximum Drawdown

Let $H_t = \max_{\tau \leq t} V_\tau$ represent the peak portfolio valuation up to day t . The drawdown DD_t is:

$$DD_t = \frac{V_t - H_t}{H_t} \quad (16)$$

The maximum drawdown is:

$$\text{Max Drawdown} = \min_t DD_t \quad (17)$$

A.2 JavaScript Code Implementations

The following sections provide code snippets from the simulation engine to show how these mathematical formulas are implemented programmatically.

A.2.1 Rebalancing Calendar Solver

The calendar solver detects date boundaries (months, quarters, years) to determine when rebalancing should occur:

```

1 // Identify Rebalancing Dates
2 let rebalanceIndices = [0];
3 for (let d = 1; d < totalDays; d++) {
4   const prevDate = dates[d - 1];
5   const currDate = dates[d];
6
7   const prevYear = prevDate.substring(0, 4);
8   const currYear = currDate.substring(0, 4);
9   const prevMonth = prevDate.substring(5, 7);
10  const currMonth = currDate.substring(5, 7);
11
12  let shouldRebalance = false;
13
14  if (params.rebalanceFreq === 'monthly') {
15    shouldRebalance = (prevMonth !== currMonth);
16  } else if (params.rebalanceFreq === 'quarterly') {
17    const prevQ = Math.floor((parseInt(prevMonth) - 1) / 3);
18    const currQ = Math.floor((parseInt(currMonth) - 1) / 3);
19    shouldRebalance = (prevQ !== currQ);
20  } else if (params.rebalanceFreq === 'annually') {
21    shouldRebalance = (prevYear !== currYear);
22  }
23
24  if (shouldRebalance) {
25    rebalanceIndices.push(d);
26  }
27 }

```

Listing 1. Rebalancing Date Resolver in app.js

A.2.2 Dogs of the Dow Trailing Dividend Yield Solver

Yields are computed by looking back at the preceding 365 days of dividend events for each stock relative to the current rebalance close price:

```

1 let yields = [];
2 tickers.forEach(ticker => {
3   const divEvents = dividends[ticker] || [];
4   const currentClose = closePrices[ticker][d];
5
6   // Sum dividends in last 365 days
7   const rebalanceDate = new Date(currentDateStr);
8   const oneYearAgo = new Date(rebalanceDate.getTime() - (365 * 24 * 60 * 60 * 1000));
9
10  let divSum = 0;
11  divEvents.forEach(e => {
12    const ed = new Date(e.date);
13    if (ed >= oneYearAgo && ed <= rebalanceDate) {
14      divSum += e.amount;
15    }
16  });
17
18  const divYield = currentClose > 0 ? divSum / currentClose : 0;

```

```

19   yields.push({ ticker, yield: divYield });
20 });
21 yields.sort((a, b) => b.yield - a.yield);
22 selectedAssets = yields.slice(0, params.stockCount).map(y => y.ticker);

```

Listing 2. Trailing Dividend Yield Calculation in app.js

A.2.3 Transaction Cost and Order Solver

Calculates trade adjustments, applies costs, and solves for new share allocations:

```

1  // Compute target and current allocations
2  let targetAllocations = {};
3  tickers.forEach(ticker => {
4    targetAllocations[ticker] = weights[ticker] || 0;
5  });
6  let currentAllocations = {};
7  tickers.forEach(ticker => {
8    const stockVal = shares[ticker] * prices[ticker][d];
9    currentAllocations[ticker] = totalValue > 0 ? stockVal / totalValue : 0;
10 });
11 // Calculate trade differences
12 let tradesToExecute = [];
13 let totalTradeValue = 0;
14 tickers.forEach(ticker => {
15   const targetValue = totalValue * targetAllocations[ticker];
16   const currentValue = shares[ticker] * prices[ticker][d];
17   const tradeDiff = targetValue - currentValue;
18
19   if (Math.abs(tradeDiff) > 1.0) {
20     tradesToExecute.push({
21       ticker,
22       currentValue,
23       targetValue,
24       diff: tradeDiff,
25       price: prices[ticker][d]
26     });
27     totalTradeValue += Math.abs(tradeDiff);
28   }
29 });
30 // Deduct costs and allocate capital
31 const totalTransactionCosts = totalTradeValue * params.transactionCost;
32 const investableCapital = totalValue - totalTransactionCosts;
33 let newShares = {};
34 tickers.forEach(ticker => {
35   const tickerTargetVal = investableCapital * targetAllocations[ticker];
36   newShares[ticker] = prices[ticker][d] > 0 ? tickerTargetVal / prices[ticker][d] : 0;
37 });
38 shares = newShares;
39 let finalStockValue = 0;
40 tickers.forEach(ticker => {
41   finalStockValue += shares[ticker] * prices[ticker][d];
42 });
43 cash = totalValue - finalStockValue - totalTransactionCosts;

```

Listing 3. Order Ingestion and Execution Cost solver in app.js

A.2.4 Entrant and Exit Composition Card Appender

Compares target portfolios between rebalance index r and $r - 1$ to compute composition differences during playback:

```

1  function renderDynamicCompositionCard(rebalanceIdxIdx) {

```

```

2   const rebalanceIndices = backtestResults.rebalanceIndices;
3   const dailySnapshots = backtestResults.dailySnapshots;
4
5   const currIdx = rebalanceIndices[rebalanceIdxIdx];
6   const currSnapshot = dailySnapshots[currIdx];
7   const currAssets = currSnapshot.assets;
8   const date = currSnapshot.date;
9   const portfolioVal = currSnapshot.value;
10
11  let prevAssets = [];
12  if (rebalanceIdxIdx > 0) {
13      const prevIdx = rebalanceIndices[rebalanceIdxIdx - 1];
14      prevAssets = dailySnapshots[prevIdx].assets;
15  }
16
17  // Set operations for composition differences
18  const entrants = currAssets.filter(t => !prevAssets.includes(t));
19  const exits = prevAssets.filter(t => !currAssets.includes(t));
20
21  const card = document.createElement('div');
22  card.className = 'timeline-event-card';
23
24  // Construct HTML elements
25  const node = document.createElement('div');
26  node.className = 'timeline-node';
27  card.appendChild(node);
28
29  const header = document.createElement('div');
30  header.className = 'timeline-event-header';
31  header.innerHTML = `
32      <span class="timeline-event-date">${date}</span>
33      <span class="timeline-event-value">${Math.round(portfolioVal).toLocaleString()}</span>
34  `;
35  card.appendChild(header);
36
37  const body = document.createElement('div');
38  body.className = 'timeline-event-body';
39
40  // Entrants list rendering
41  const entrantsCol = document.createElement('div');
42  entrantsCol.className = 'timeline-column';
43  const entrantsListHTML = entrants.length === 0
44      ? '<span class="empty-column-msg">None</span>'
45      : entrants.map(t => '<span class="ticker-badge entrant">${t}</span>').join('');
46  entrantsCol.innerHTML = `
47      <h4>Joined</h4>
48      <div class="timeline-column-list">${entrantsListHTML}</div>
49  `;
50  body.appendChild(entrantsCol);
51
52  // Exits list rendering
53  const exitsCol = document.createElement('div');
54  exitsCol.className = 'timeline-column';
55  const exitsListHTML = exits.length === 0
56      ? '<span class="empty-column-msg">None</span>'
57      : exits.map(t => '<span class="ticker-badge exit">${t}</span>').join('');
58  exitsCol.innerHTML = `
59      <h4>Left</h4>
60      <div class="timeline-column-list">${exitsListHTML}</div>
61  `;
62  body.appendChild(exitsCol);
63  card.appendChild(body);
64
65  // Prepend to timeline
66  DOM.timelineFeed.insertBefore(card, DOM.timelineFeed.firstChild);
67  }

```

Listing 4. Dynamic Entrant and Exit Badge Appender in app.js